

HITO 1 · 08 DE JUNIO, 2026

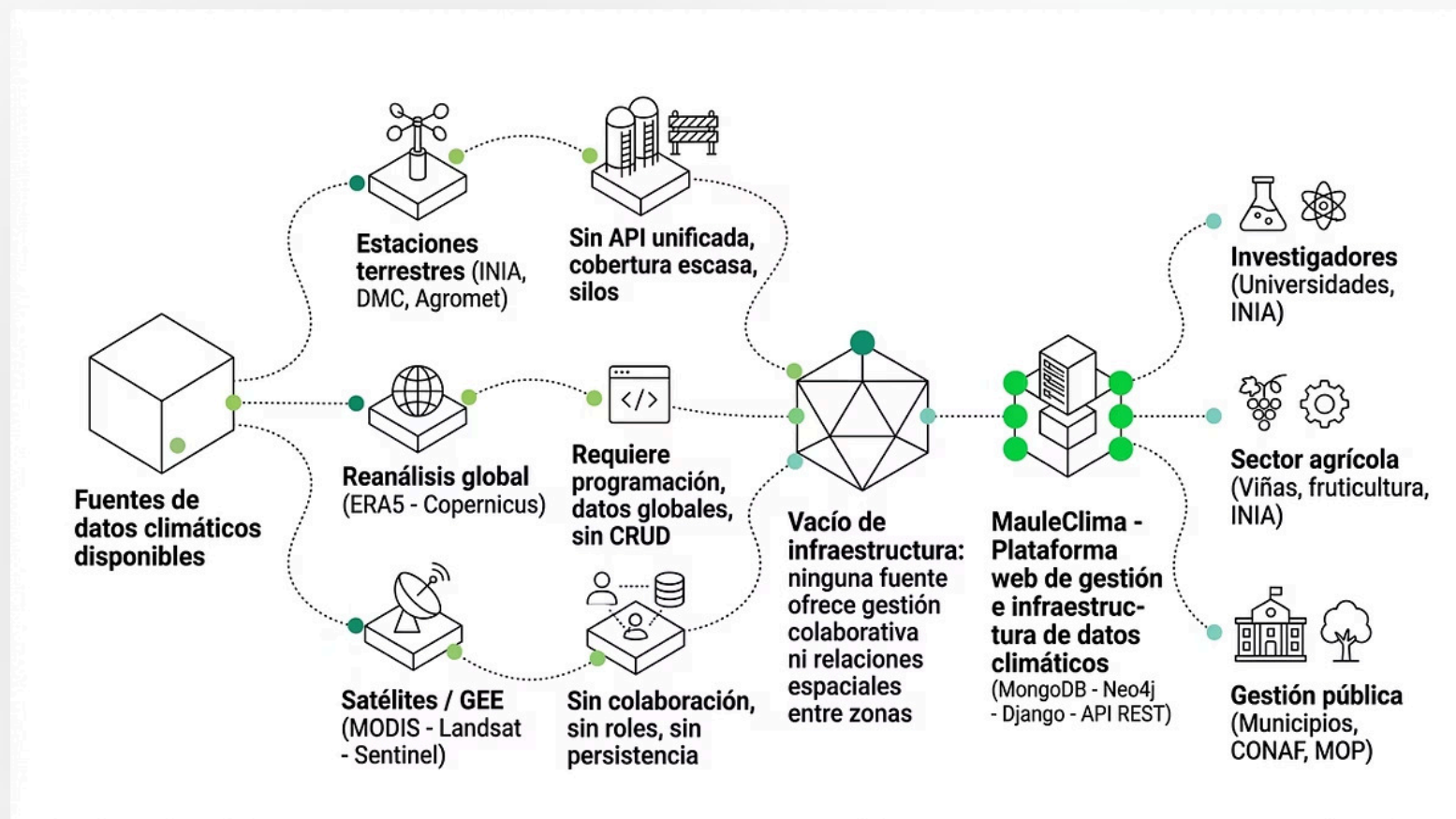
MauleClima

Plataforma de gestión y exploración colaborativa de datos climáticos satelitales de la Región del Maule

Benjamín Vera Garrido · Bases de Datos Avanzadas

Descripción del problema

Los grupos de investigación climática en la Región del Maule operan hoy con datos dispersos, procesos manuales y sin una base común para compartir ni consultar información. El resultado: trabajo no reproducible, tiempo perdido en integración y dificultad para responder preguntas complejas sobre el territorio.



Problemáticas de obtención de datos

Estaciones terrestres (ej: INIA, DMC, Agromet)

Aparatos con sensores puestos en el suelo, como termómetros y pluviómetros

1. Hay muy pocas para toda la región (como 30 estaciones para 30.000 km²)
2. Cada estación tiene su propia página web
3. No hay una API

Modelos de reanálisis global (ERA5, MODIS)

Combinan observaciones reales (satélites/estaciones) con leyes de la física para reconstruir el clima continuo

ERA5 (modelo de Copernicus) da precipitación, humedad, viento y temperatura del aire. **MODIS** es un instrumento. Fotos que calcula la temperatura de la superficie y un índice de vegetación (NDVI).

1. Requiere programar
2. Formatos complejos para el análisis de datos (NetCDF, HDF).
3. Son globales
4. Sin CRUD

Google Earth Engine (GEE)

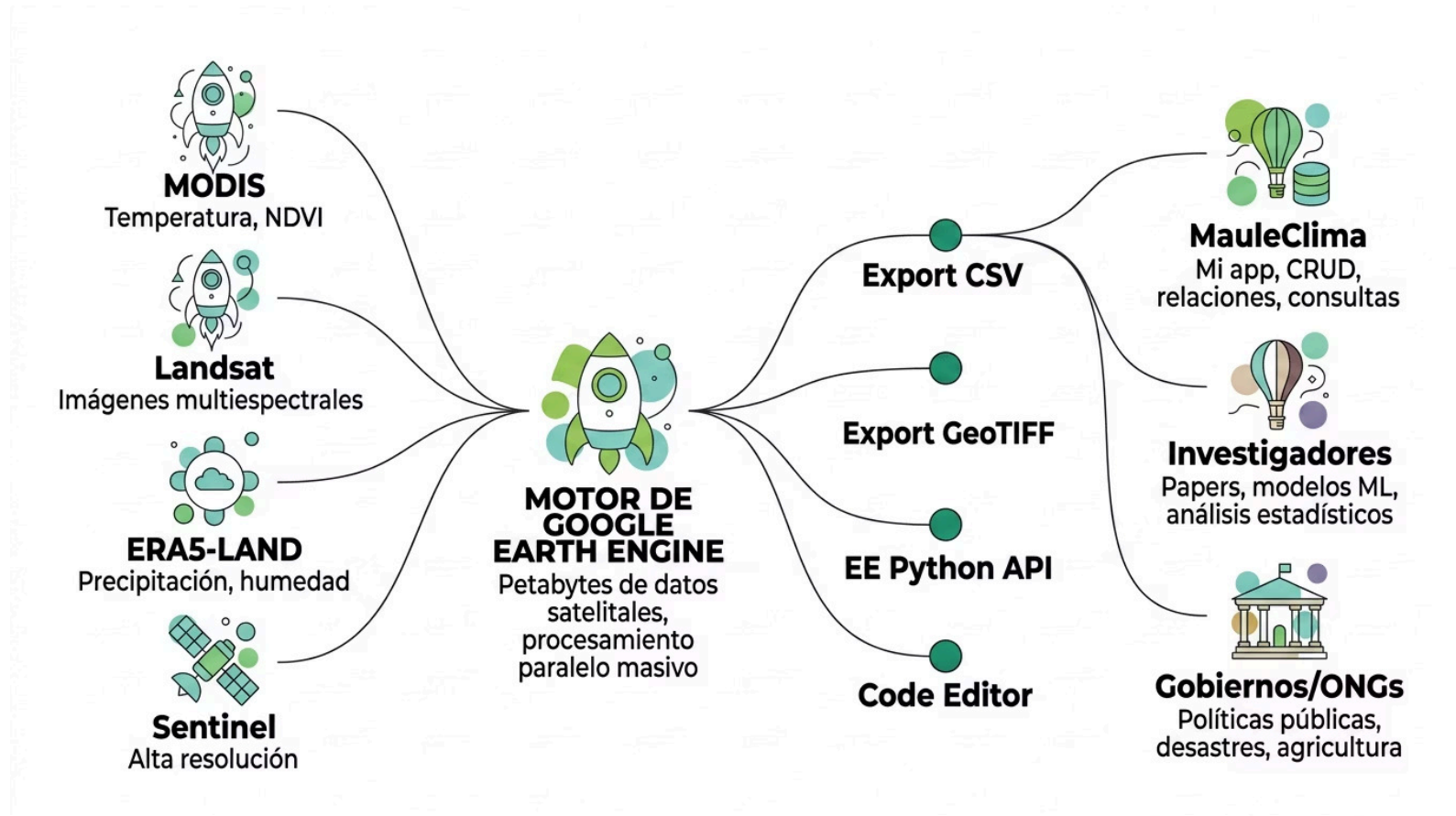
Procesador en la nube de Google. Tiene servidores con datos de ERA5, MODIS, Landsat y cientos de satélites más desde los años 80.

Acepta consultas como "T° promedio mensual por comuna Maule entre 2020 y 2023" y te devuelve archivo .CSV

1. No tiene CRUD
2. No hace relaciones espaciales
3. Solo para análisis no para archivar datos.
4. Bloquea API

Ecosistema de Google Earth Engine

Comprender el rol de Google Earth Engine (GEE) es clave para la gestión de datos climáticos. Esta infografía desglosa su funcionamiento, desde las fuentes de datos que procesa hasta las diversas aplicaciones que potencia, incluyendo nuestra plataforma MauleClima.

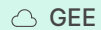


¿Quién usa GEE y para qué?

Propuesta de solución

GEE es una herramienta de producción de datos y **MauleClima** es una herramienta de gestión, consulta y colaboración sobre esos datos. **Es una capa adicional, no busca competir con GEE.**

1



Producción de datos

Procesa información satelital y ejecuta scripts para generar exportaciones.

2



Gestión y colaboración

Organiza, consulta y comparte datos con trazabilidad y roles de usuario.



Limitaciones de GEE

- Sin cuentas colaborativas con roles
- Sin trazabilidad de quién exporta un CSV ni con qué versión de script
- Sin CRUD
- No hace consultas complejas ni relaciona datos
- Requiere JavaScript, Python y scripts manuales



Aportes de MauleClima

- Cuentas colaborativas con roles
- Trazabilidad completa
- CRUD integrado
- Consultas complejas y relaciones
- Interfaz web intuitiva

Si alguien quiere modificar un script, en GEE deberá editarlo o escribir otro, ejecutarlo, esperar, exportar, descargar y luego analizar el CSV. MauleClima simplifica ese flujo.



Objetivo general

Diseñar e implementar una aplicación web para la gestión, consulta y exploración colaborativa de datos climáticos satelitales de la Región del Maule, integrando bases de datos **NoSQL documentales y de grafos** para modelar tanto las mediciones temporales como las relaciones espaciales entre zonas geográficas.

Objetivos específicos

1

Modelado de datos climáticos

Documentos climáticos en MongoDB y relaciones geoespaciales como grafo en Neo4j.

2

Backend con API REST

Django con operaciones CRUD completas para zonas y mediciones climáticas.

3

Frontend funcional

Registro, autenticación, rutas protegidas y validación de formularios.

4

Consultas en Cypher

Preguntas como: ej. "zonas vecinas a Talca con temperatura > 30°C en enero 2023".

5

Consistencia referencial

Integración de ambas bases manteniendo coherencia entre IDs de MongoDB y nodos en Neo4j.

6

Visualización del grafo

Relaciones espaciales en el frontend mediante vis.js o D3.js.

Funcionalidades principales (1/2)



F1 · Autenticación

Registro y login con roles diferenciados: **admin**, **editor** y **viewer**. Protección de rutas según permisos.



F2 · Mapa interactivo

Mapa de la Región del Maule con las 30 comunas. Al hacer clic, popup con temperatura, precipitación y NDVI.



F3 · Explorador de grafos

Visualización de comunas y sus relaciones de vecindad y toografía. Navegación interactiva con datos climáticos por nodo.



F4 · Consultas

Lista de preguntas del dominio climático. Al seleccionar una, tabla con comunas y valores que cumplen la condición.

Funcionalidades principales (2/2)



F5 · Panel de administración

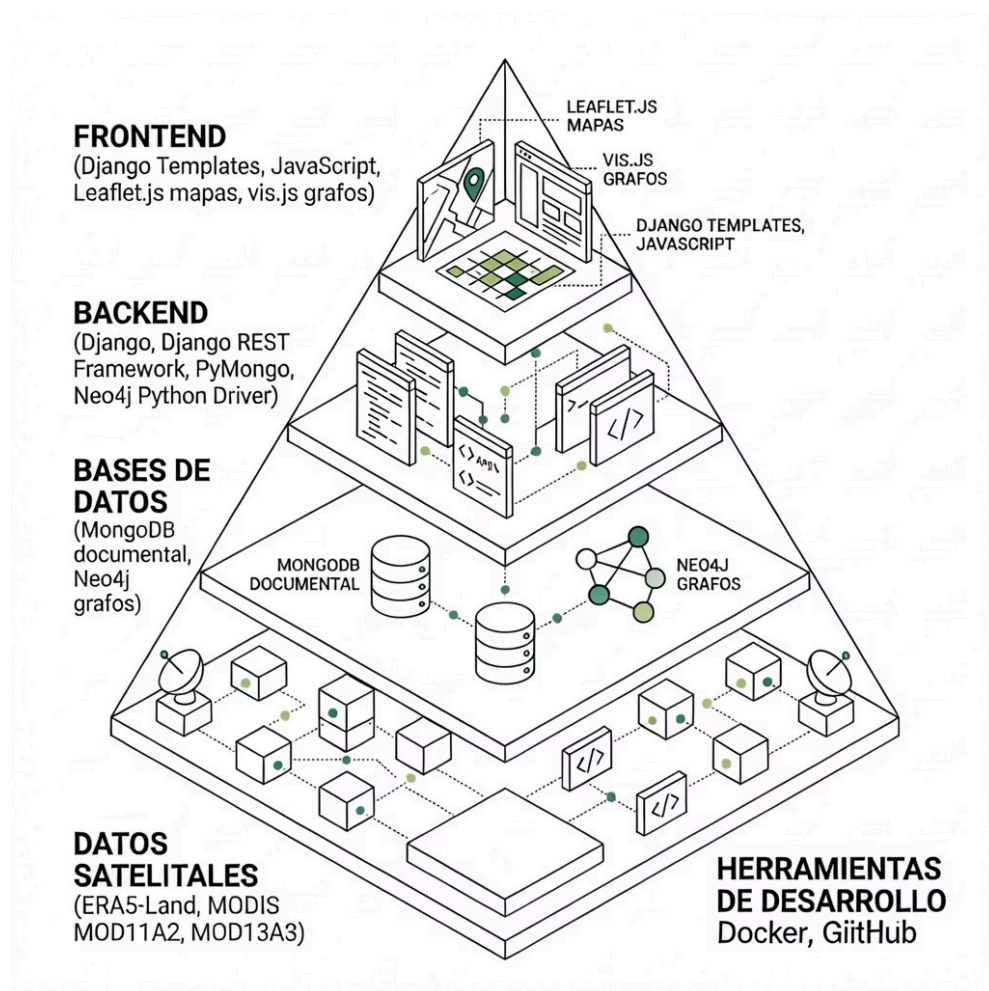
Importación de CSV exportados de Google Earth Engine. Edición y eliminación de zonas y mediciones en MongoDB y Neo4j.



F6 · API REST

Rutas como `/api/zonas/`, `/api/mediciones/` y `/api/consultas/`. Datos en JSON, accesibles vía Post o el frontend.

Tecnologías a utilizar



Stack tecnológico

La arquitectura combina lo mejor de dos modelos NoSQL: **MongoDB** para almacenar mediciones temporales como documentos flexibles, y **Neo4j** para modelar y consultar relaciones espaciales entre comunas.

- **Backend:** Django + Django REST Framework
- **Datos satelitales:** ERA5-Land y MODIS desde Google Earth Engine
- **Desarrollo:** Docker y GitHub para control de versiones

Estructura de datos en MongoDB

Tres colecciones documentales modelan la información del sistema:

Usuario

Almacena credenciales, nombre y rol (admin, editor, viewer) con marca de tiempo de creación.

Zonas (Comuna)

Identificador único, nombre, provincia, coordenadas, altitud y usuario creador.

Mediciones

Registros climáticos mensuales por zona: temperatura diurna/nocturna, NDVI, precipitación y humedad.

- ❗ Los IDs de zona en MongoDB se sincronizan con los nodos en Neo4j para garantizar **consistencia referencial** entre ambas bases de datos.

Estructura de datos en MongoDB

Tres colecciones documentales modelan la información del sistema:

Usuario

```
{
  "_id": "ObjectId",
  "email": "investigador@mail.cl",
  "password_hash": "bcrypt...",
  "nombre": "Ana García",
  "rol": "admin",
  "created_at": "2024-01-15T10:00:00Z"
}
```

Zonas (Comuna)

```
{
  "_id": "ObjectId",
  "zona_id": "talca_01",
  "nombre": "Talca",
  "provincia": "Talca",
  "lat": -35.4264,
  "lon": -71.6554,
  "altitud_m": 102,
  "created_by": "ObjectId(user)"
}
```

Mediciones

```
{
  "_id": "ObjectId",
  "zona_id": "talca_01",
  "fecha": "2023-01",
  "fuente": "MODIS+ERA5",
  "lst_dia_c": 34.2,
  "lst_noche_c": 18.1,
  "precip_mm": 12.3,
  "humedad_pct": 48.0,
  "created_by": "ObjectId(user)"
}
```

Estructura de datos en Neo4j

Nodo: Zona

Representa cada comuna como un nodo. Solo tiene en común con una colección los la zona o comuna

```
(:Zona {  
  zona_id: "talca_01",  
  nombre: "Talca",  
  lat: -35.4264,  
  lon: -71.6554,  
  cuenca: "Rio Maule"  
})
```

Relaciones VECINO_DE

Modela qué comunas comparten borde geográfico

```
(:Zona {zona_id:"talca_01"})-[:VECINO_DE {  
  distancia_km: 42.3  
}]-> (:Zona {zona_id:"san_clemente_01"})
```

Relación CLIMA_SIMILAR

Conecta comunas con historial climático parecido. ¿qué comunas se comportan igual?

```
(:Zona {zona_id:"talca_01"})-[:CLIMA_SIMILAR {  
  similitud_temp: 0.91,  
  similitud_ndvi: 0.87 }]  
-> (:Zona {zona_id:"curico_01"})
```

Relación COMPARTE_TOPOGRAFIA

Conecta comunas que drenan al mismo río (por ejemplo). ¿Que comunas estan rio arriba de Constitución?

```
(:Zona {zona_id:"talca_01"})-[:COMPARTE_CUENCA {  
  nombre_cuenca: "Rio Maule"  
}]-> (:Zona {zona_id:"constitucion_01"})
```

Plan de trabajo y cronograma

Semana	Tareas	OE s cubiertos	Entregable / justificación
1	Configuración del entorno: instalar Django, MongoDB, Neo4j, crear proyecto. Diseño inicial del modelo de datos (MongoDB y Neo4j).	OE1	Entorno funcionando, modelos definidos.
1	Implementar autenticación (registro, login, logout, roles básicos). Proteger rutas.	OE3	Usuarios pueden registrarse y acceder.
2-3	Crear el CRUD de zonas y mediciones en MongoDB desde Django (vistas, formularios). Cargar datos iniciales (CSV).	OE2, OE5	Admin puede gestionar zonas y mediciones.
3-4	Desarrollar el frontend: mapa (Leaflet) y explorador de grafos (vis.js).	OE3, OE6	Pantallas Mapa y Grafos interactivas.
5	Implementar la funcionalidad de Consultas predefinidas (con Cypher + MongoDB). Probar integración híbrida. Corregir errores, documentar todo para la presentación final.	OE4, OE5	Demostración completa de todos los requisitos.

HITO 1 · 08 DE JUNIO, 2026

MauleClima – Fin de la presentación

Plataforma de gestión y exploración colaborativa de datos climáticos satelitales de la Región del Maule

Benjamín Vera Garrido · Bases de Datos Avanzadas